URBAN MOBILITY DATA EXPLORER

TECHNICAL REPORT | SUMMATIVE ASSESSMENT

Turning millions of New York City taxi trips into clear, useful mobility information.

TEAM

Sam : Frontend dashboard

Devis : Data cleaning and algorithms

Ivan : Backend and database

Project goal: clean and validate taxi-trip data, store it in a structured database, calculate mobility summaries, and present the results through an interactive dashboard.

Dataset: NYC Yellow Taxi trip records, January 2019

The report follows the five assessment requirements: problem framing,architecture,a manually implemented algorithm, three data insights, and reflection.

Executive summary

Urban Mobility Data Explorer helps users explore taxi demand, trip locations, travel times, distances, and fares. The team built a data cleaning pipeline,validation rules,a custom top routes algorithm,a MySQL relational schema,a Flask API and a browser-based dashboard. The dashboard turns detailed trip records into summary cards, charts, filters, and trip-level information.

7,667,792	112,310	7,555,482	1.46%
Input records	Removed records	Retained records	Removed share

The cleaning output contains 7,555,482 records and 30 columns. The saved validation report records zero failures for the implemented checks. These results show why validation is important before charts or route rankings are produced.

Contents

1. Problem framing and dataset analysis • 2. System architecture and design decisions • 3. Algorithmic logic and data structures • 4. Insights and interpretation • 5. Reflection and future work

---

1. Problem framing and dataset analysis

The problem

Taxi trip records can show where travel demand is concentrated, when people travel, and how distance relates to cost. Raw data can also contain missing values, incorrect timestamps, impossible measurements, or inconsistent category codes. If these issues are not handled, the dashboard may present misleading averages and rankings. Our goal was to create a reliable path from raw records to useful mobility summaries.

Dataset context

The dataset contains New York City Yellow Taxi trips from January 2019. Each row describes a trip. Important fields include pickup and drop-off timestamps, pickup and drop-off zone IDs, trip distance, passenger count, fare, tip, tolls, payment type, and total amount. A zone lookup connects location IDs to names and boroughs.

Main data challenges

Challenge	What we found	How we handled it
Missing fees	Airport fee missing in 100% of rows; congestion surcharge missing in 63.33%.	Standardised fee fields for processing and documented the assumption that missing is not always the same as zero.
Missing coordinates	Pickup coordinates missing in 2.13%; drop-off coordinates missing in 2.17%.	Used zone references where available and left unavailable zone coordinates empty instead of inventing locations.
Time anomalies	6,294 trips were flagged with drop-off not after pickup; 74,069 trips were shorter than one minute.	Applied time-order and duration rules. A row may trigger more than one rule, so flag totals are not added together.

Extreme or invalid values	Some records had invalid distance, speed, fare, total amount, or passenger counts.	Applied documented range checks and excluded records that failed critical rules.

Cleaning assumptions

We accepted trips with duration from 1 to 180 minutes, distance above 0 and no more than 100 miles, average speed no higher than 80 mph, fare above $0 and no more than $500, total amount no higher than $1,000, and passenger count from 1 to 6. Missing or zero passenger counts in otherwise accepted rows were replaced with 1. Unknown category codes were mapped to known unknown values. Missing fee values were filled with zero so the fields could be processed consistently; this is a practical assumption and should be explained when interpreting fee charts.

Unexpected observation: the airport-fee field was missing in every original row. This influenced our design by showing that the dashboard must treat data quality as part of interpretation: a zero after cleaning can mean “filled during cleaning,” not necessarily “no fee was charged.”

2. System architecture and design decisions

Architecture

The system uses three connected layers. The frontend presents controls and visualisations. The Flask backend receives requests, validates parameters, performs the required query or calculation, and returns JSON. MySQL stores cleaned trips and reference data in related tables. The cleaning pipeline prepares and validates data before it is loaded for analysis.

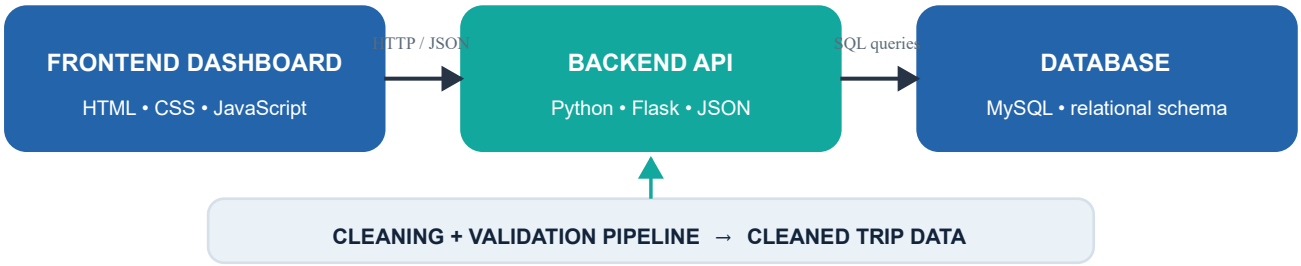


Figure 2. Main data flow from the cleaning pipeline through the database and API to the dashboard.

Technology choices

Layer	Technology	Reason for the choice
Frontend	HTML, CSS, JavaScript	Runs in a browser, supports interactive filters and charts, and can adapt to desktop, tablet, and mobile screens.
Backend	Python and Flask	Python supports data processing and custom algorithms; Flask provides a straightforward way to expose API endpoints.

Database	MySQL	Relational tables, keys, and constraints help maintain consistent relationships across millions of trips.
Data pipeline	Python cleaning and validation scripts	Repeatable rules make the preparation process easier to review and reproduce.

Database structure

The main **trips** table stores one row per cleaned trip. Reference tables store zones, boroughs, service zones, vendors, rate codes, and payment types. Foreign keys connect trips to their pickup and drop-off zones and to the relevant vendor, rate, and payment records. This reduces repeated text and helps prevent trips from referring to unknown reference records. Indexes on pickup time, location plus time, distance, and total amount support common filters, charts, and sorting.

Trade-offs

A relational database gives clear structure and integrity checks, but loading millions of rows requires careful batching and index planning. Separating the frontend, API, and database lets each team member work independently, but it also requires agreement on field names, data types, filter parameters, and API responses. We prioritised a simple API with GET endpoints returning JSON so that the dashboard can request summary cards, trip records, zone profiles, and chart data without duplicating database logic in the browser.

Key design decision: the database stores normalised reference information once, while the API presents the information in a form that is convenient for the dashboard.

3. Algorithmic logic and data structures

Custom top-k route ranking with a MinHeap

One of our manually implemented algorithms finds the most frequent taxi routes. A route is represented as a pair: (pickup zone ID, drop-off zone ID). The algorithm counts each route and then uses a custom minimum heap to keep only the k highest-count routes. It does not rely on ready-made ranking tools such as Counter, heapq, or a built-in sort operation for the main selection process.

Approach

First, read each trip and increase the count for its pickup/drop-off pair in a dictionary. Next, insert routes into a custom MinHeap until it contains k routes. After that, compare each remaining route with the smallest count in the

heap. If the new route has a larger count, remove the smallest route and insert the new one. Finally, extract the retained routes in descending count order.

Pseudocode

```
FUNCTION top_k_routes(rows, k):     counts =
empty dictionary

    FOR each trip IN rows:        route = (trip.pickup_zone,
trip.dropoff_zone)        counts[route] = counts.get(route, 0) + 1

    heap = empty custom MinHeap     FOR each
(route, count) IN counts:
        item = (count, route)        IF heap.size < k:
heap.push(item)        ELSE IF count >
heap.peek_min().count:
            heap.pop_min()            heap.push(item)

    result = empty list     WHILE heap is not
empty:        result.append(heap.pop_min())

    reverse result using a manual loop
    RETURN result
```

Complexity analysis

Symbol	Meaning
n	Number of trip records.
m	Number of unique pickup/drop-off routes.
k	Number of top routes requested.

Measure	Complexity	Explanation
Counting	Expected O(n)	Each trip updates one route count in the dictionary.
Heap selection	O(m log k)	Each unique route can trigger a heap operation taking O(log k).
Total time	Expected O(n + m log k)	Combines counting and keeping the best k routes.
Space	O(m + k)	The dictionary stores m route counts and the heap stores at most k items.

Why this algorithm is useful

A simple full sort would rank every unique route, even when the user only wants a small top-k list. A bounded heap keeps the candidate list small and makes route ranking efficient for a large dataset. The output can support a dashboard ranking of popular travel connections and help users explore where taxi movement is concentrated.

4. Insights and interpretation

The following three insights are calculated from the saved quality and cleaning summaries. Each visual shows a measurable result, explains how it was derived, and connects it to the reliability of urban-mobility analysis.

Insight 1 — The cleaning pipeline retained most trips while removing invalid records

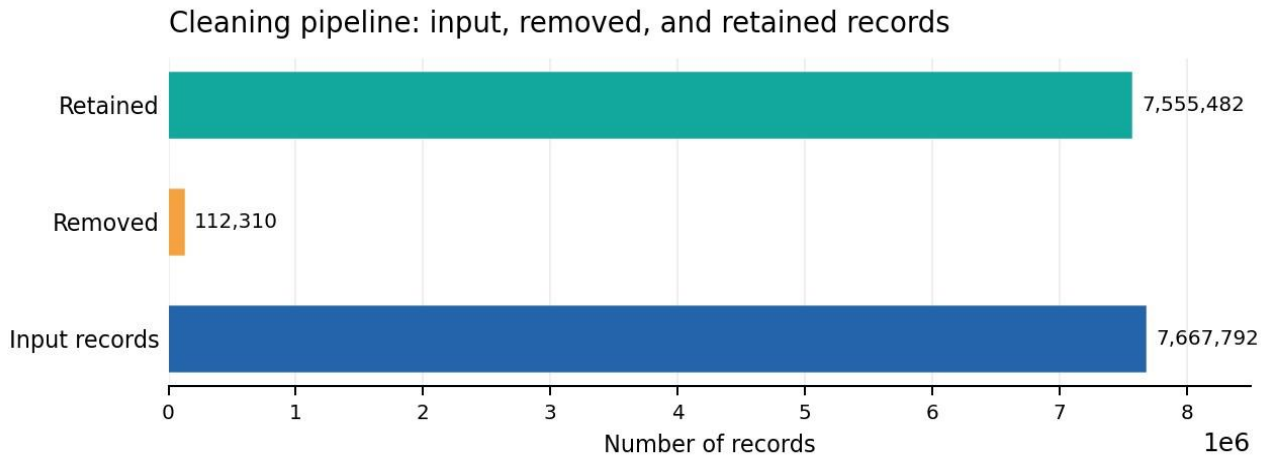


Figure 3. Record counts before and after cleaning.

How we derived it: we compared the input, removed, and retained row counts in the cleaning report. Of 7,667,792 input records, 112,310 were removed and 7,555,482 were retained. The removed share was 1.46%.

Interpretation: most trips passed the cleaning rules, but more than one hundred thousand records were excluded. Even a small percentage can affect averages, trip counts, and route rankings, so cleaning before analysis improves the trustworthiness of the dashboard.

Insight 2 — Missing fee fields are a major interpretation risk

How we derived it: we compared the missing-value percentages in the original quality report. Airport fee was missing in 100% of rows, while congestion surcharge was missing in 63.33%. Pickup and drop-off coordinates were missing at much lower rates, 2.13% and 2.17%.

Interpretation: fee-related charts need more care than location charts. Replacing missing fees with zero makes records easier to process, but it may understate charges if users interpret every zero as a confirmed value. A missingness indicator or a clear note beside fee charts is a sensible safeguard.

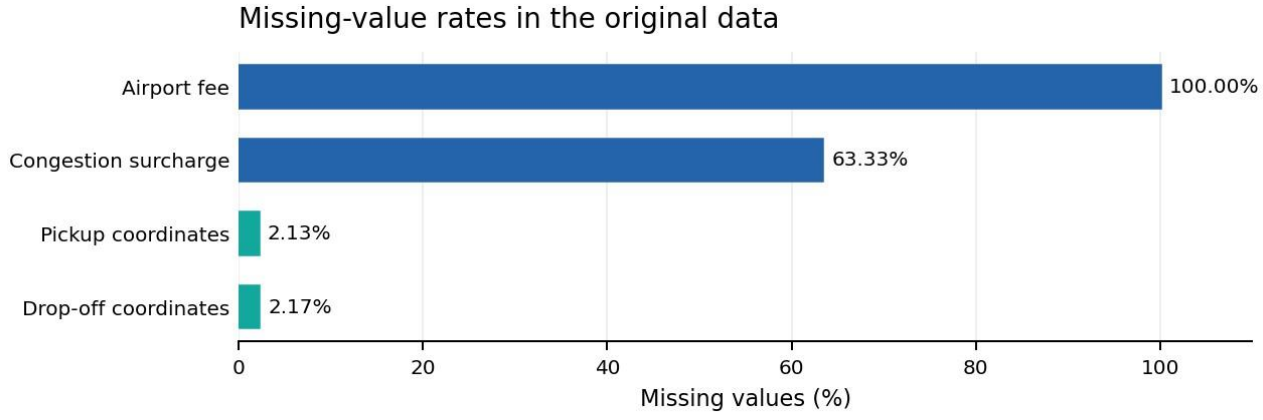


Figure 4. Comparison of missingness across selected fields.

Insight 3 — Cleaning changed the average trip duration

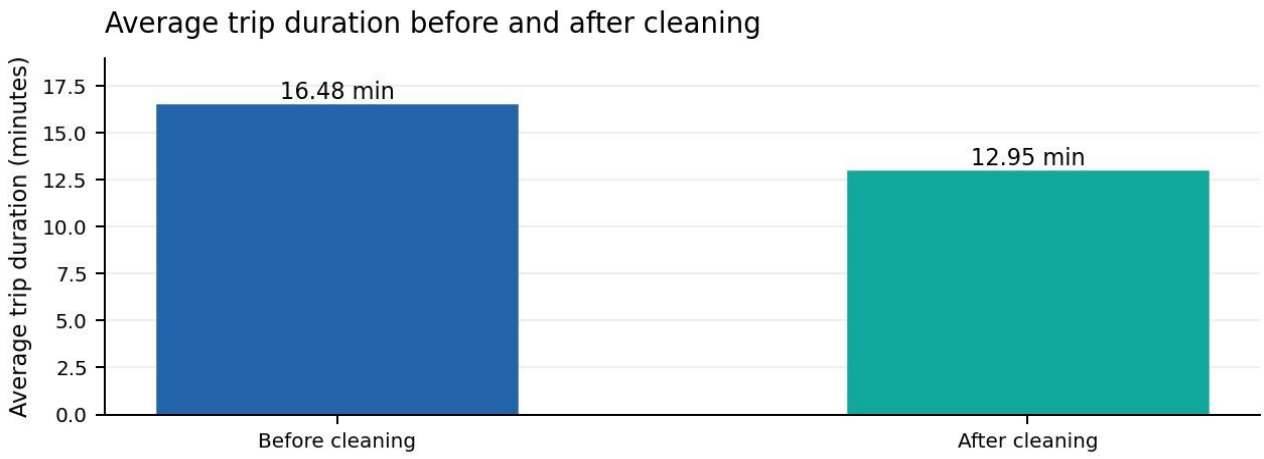


Figure 5. Average trip duration before and after cleaning.

How we derived it: the before/after summary reports a mean duration of 16.481 minutes before cleaning and 12.953 minutes after cleaning, a decrease of about 3.53 minutes.

Interpretation: invalid timestamps and unusual trip durations can raise or distort averages. The change shows that the cleaning rules materially affect the summary statistics that users see. It does not mean every trip became shorter; it means the average of the retained records is different after invalid or implausible records are removed.

How the dashboard supports further exploration

The dashboard and API are designed to let users examine trips by date, pickup or drop-off zone, borough, distance range, fare range, and payment type. Summary cards and charts for hourly activity, daily activity, boroughs, top zones, payment types, and distance distribution help users move from an overall view to more specific questions. The custom route-ranking algorithm adds a direct way to identify frequently used zone-to-zone connections.

5. Reflection and future work

Teamwork and technical challenges

The work was divided by responsibility: Devis focused on data cleaning, validation, and algorithms; Ivan focused on the backend and database; Sam focused on the frontend dashboard. This division allowed us to work on different parts in parallel. The main teamwork challenge was keeping the layers consistent: field names, API parameters, database types, and chart expectations needed to match. The data challenge was deciding how to treat missing values and outliers without introducing false information.

What we learned

We learned that a dashboard is only as useful as the data and rules behind it. A clear cleaning pipeline makes analysis repeatable, relational constraints help protect data quality, and a custom algorithm demonstrates how algorithmic thinking can solve a real problem. We also learned to explain assumptions in simple language, because technical decisions can change how users interpret the results.

Future improvements

Priority	Improvement	Benefit
1	Preserve a separate flag for fee values that were originally missing.	Users can distinguish unknown charges from confirmed zero charges.
2	Add a data-quality panel with missingness and cleaning counts.	Users can see how data preparation affects results.
3	Expand route analysis with time-of-day and weekday comparisons.	Reveals whether popular connections change across periods.
4	Improve map-based exploration and zone profiles.	Makes geographic patterns easier to understand.
5	Add performance monitoring and repeatable automated tests.	Helps maintain stable results as data volume or code changes.

Conclusion

Urban Mobility Data Explorer brings together data preparation, a structured database, a Flask API, a dashboard, and a manually implemented route-ranking algorithm. The January 2019 taxi data shows that cleaning removed 1.46% of records, missing fee values require careful interpretation, and cleaning can materially change average trip duration. Together, these results demonstrate why reliable data handling and transparent algorithmic choices matter when building tools for urban mobility analysis.

Final takeaway: a good mobility dashboard does more than display numbers. It makes the data easier to explore, explains how results were produced, and helps users interpret patterns responsibly.

Project components referenced in this report: data cleaning and validation scripts/logs, custom algorithm and tests, MySQL schema and indexes, Flask API endpoints, and frontend dashboard files.97